DHSK College

Department of Computer Science

BCA – Semester IV

Data Structures

Practical Assignment Booklet

Subject	Data Structures (BCA – Sem IV)
Units Covered	Unit 1 (Arrays, Searching, Sorting) & Unit 2 (Linked Lists, Stacks, Queues)
Programming Language	C
Total Practicals	12 (including 1 Mini Project)
Submission Mode	Google Classroom: (1) Handwritten program – scanned/photographed, (2) Screenshot of output, (3) Viva answers in Word/PDF
Submission Deadline	To be announced on Google Classroom
Assigned by	HOD, Department of Computer Science, DHSK College

Instructions for Students

- Each student must write all programs individually in their practical notebook. Group submissions will not be accepted.
- Write the program neatly by hand in your practical notebook. Clearly write the Practical number, Aim, Code, and Output.
- Scan or clearly photograph each handwritten practical page and upload it as a PDF or image file.
- Run the program on a computer, take a clear screenshot of the output, and upload it along with your handwritten copy.
- Answer all Viva Questions at the end of each practical in a Word or PDF document and upload it as a single combined file named: VivaAnswers_RollNo.pdf

- Naming convention: Practical1_Scan_RollNo.pdf, Practical1_Output_RollNo.png, VivaAnswers_RollNo.pdf
- Plagiarism in handwriting or viva answers will result in cancellation of the entire assignment. Write in your own words.
- Deadline is strictly enforced. Late submissions will not be accepted unless prior permission is obtained.
- For any queries, post a comment on Google Classroom — do not message personally.

Practical List & Marks Distribution

No.	Title	Topic / Unit	Marks
1	Array Operations – Insertion, Deletion & Traversal	Arrays – Unit 1	5
2	Searching – Linear Search & Binary Search	Arrays / Searching – Unit 1	5
3	Sorting – Bubble Sort & Selection Sort	Arrays / Sorting – Unit 1	5
4	Singly Linked List – Create, Insert & Delete	Linked Lists – Unit 2	5
5	Doubly Linked List – Insert & Delete	Linked Lists – Unit 2	5
6	Stack Implementation Using Array	Stacks – Unit 2	5
7	Stack Application – Parenthesis Matching	Stacks / Applications – Unit 2	5
8	Queue Implementation Using Array	Queues – Unit 2	5
9	Circular Queue Implementation	Queues – Unit 2	5

No.	Title	Topic / Unit	Marks
10	Stack Using Linked List	Stacks / Linked Lists – Unit 2	5
11	Queue Using Linked List	Queues / Linked Lists – Unit 2	5
12	Mini Project – Student Record Management Using Linked List	Arrays / Linked Lists (Integration) – Unit 1 & 2	10

Total Marks: 65 (Practicals 1–11: 5 marks each = 55 | Practical 12 Mini Project: 10 marks)

Practical 1: Array Operations – Insertion, Deletion & Traversal

Unit: Unit 1 | Topic: Arrays

Aim	Write a C program to perform insertion, deletion, and traversal operations on a 1D array.

Objectives

- Understand array declaration and initialization in C.
- Implement insertion of an element at a given position.
- Implement deletion of an element from a given position.
- Display the array before and after each operation.

Steps to Follow

1. Declare an integer array of size 10 and take input from the user.
2. Display the original array.
3. Insert an element at a user-specified position.
4. Delete an element from a user-specified position.
5. Display the final array after both operations.

Reference Code (Study & Write in Notebook by Hand)

Note: Do NOT copy-paste. Understand the logic and write the complete program in your own handwriting.

```
#include<stdio.h>
#define MAX 10
void display(int a[], int n) {
    for(int i=0; i<n; i++) printf("%d ", a[i]);
    printf("\n");
}
void insert(int a[], int *n, int pos, int val) {
    for(int i=*n; i>pos; i--) a[i]=a[i-1];
    a[pos]=val; (*n)++;
}
void delete(int a[], int *n, int pos) {
    for(int i=pos; i<*n-1; i++) a[i]=a[i+1];
```

```
    (*n)--;
}
int main() {
    int a[MAX], n;
    printf("Enter n: "); scanf("%d",&n);
    for(int i=0;i<n;i++) scanf("%d",&a[i]);
    display(a,n);
    // Insert and delete as per user input
    return 0;
}
```

Expected Output

Show array before insertion, after insertion, and after deletion.

Viva Questions

Q1. What is the time complexity of insertion at the beginning of an array?

Q2. Why does deletion at position 0 require shifting all elements?

Practical 2: Searching – Linear Search & Binary Search

Unit: Unit 1 | Topic: Arrays / Searching

Aim	Implement and compare Linear Search and Binary Search algorithms on an integer array.

Objectives

- Understand the concept of searching in arrays.
- Implement Linear Search and Binary Search in C.
- Count and compare the number of comparisons made by each algorithm.
- Understand when Binary Search requires sorted data.

Steps to Follow

6. Create an array of 10 integers.
7. Implement Linear Search and display the number of comparisons.
8. Sort the array (use Bubble Sort or built-in sort).
9. Implement Binary Search on the sorted array.
10. Display whether the element was found and how many comparisons were needed.

Reference Code (Study & Write in Notebook by Hand)

Note: Do NOT copy-paste. Understand the logic and write the complete program in your own handwriting.

```
#include<stdio.h>
int linearSearch(int a[], int n, int key) {
    for(int i=0; i<n; i++)
        if(a[i]==key) return i;
    return -1;
}
int binarySearch(int a[], int n, int key) {
    int lo=0, hi=n-1;
    while(lo<=hi) {
        int mid=(lo+hi)/2;
        if(a[mid]==key) return mid;
        else if(a[mid]<key) lo=mid+1;
        else hi=mid-1;
    }
    return -1;
}
```

Expected Output

Position of the element in the array, and number of comparisons for each method.

Viva Questions

Q1. What is the best and worst case time complexity of Binary Search?

Q2. When would you prefer Linear Search over Binary Search?

Practical 3: Sorting – Bubble Sort & Selection Sort

Unit: Unit 1 | Topic: Arrays / Sorting

Aim	Write a C program to sort an array using Bubble Sort and Selection Sort techniques.

Objectives

- Implement Bubble Sort and observe the pass-by-pass sorting process.
- Implement Selection Sort and compare it with Bubble Sort.
- Print the array after each pass to understand the algorithm.

Steps to Follow

11. Accept an array of n integers from the user.
12. Sort using Bubble Sort and display array after each pass.
13. Reset the array and sort using Selection Sort.
14. Compare the total number of swaps in both methods.

Reference Code (Study & Write in Notebook by Hand)

Note: Do NOT copy-paste. Understand the logic and write the complete program in your own handwriting.

```
#include<stdio.h>
void bubbleSort(int a[], int n) {
    for(int i=0; i<n-1; i++)
        for(int j=0; j<n-i-1; j++)
            if(a[j]>a[j+1]) { int t=a[j]; a[j]=a[j+1]; a[j+1]=t; }
}
void selectionSort(int a[], int n) {
    for(int i=0; i<n-1; i++) {
        int min=i;
        for(int j=i+1; j<n; j++)
            if(a[j]<a[min]) min=j;
        int t=a[i]; a[i]=a[min]; a[min]=t;
    }
}
```

Expected Output

Sorted array and number of swaps for each sorting method.

Viva Questions

Q1. What is the time complexity of Bubble Sort in the worst case?

Q2. Which sorting algorithm is better in terms of number of swaps — Bubble or Selection?

Practical 4: Singly Linked List – Create, Insert & Delete

Unit: Unit 2 | Topic: Linked Lists

Aim	Write a C program to create a singly linked list and perform insertion and deletion operations.

Objectives

- Understand the concept of dynamic memory allocation using malloc().
- Create a singly linked list and traverse it.
- Insert a node at the beginning, end, and a specific position.
- Delete a node from the beginning and by value.

Steps to Follow

15. Define a node structure with data and a next pointer.
16. Write functions: createNode(), insertAtEnd(), insertAtBeginning(), deleteNode(), display().
17. Create a linked list with at least 5 nodes.
18. Insert a node at position 2.
19. Delete a node by value and display the updated list.

Reference Code (Study & Write in Notebook by Hand)

Note: Do NOT copy-paste. Understand the logic and write the complete program in your own handwriting.

```
#include<stdio.h>
#include<stdlib.h>
struct Node { int data; struct Node* next; };
struct Node* createNode(int val) {
    struct Node* n = (struct Node*)malloc(sizeof(struct Node));
    n->data = val; n->next = NULL;
    return n;
}
void display(struct Node* head) {
    while(head) { printf("%d -> ", head->data); head=head->next; }
    printf("NULL\n");
}
```

Expected Output

Linked list after creation, after insertion, and after deletion.

Viva Questions

Q1. What is the advantage of a linked list over an array?

Q2. Why is deletion at the end of a singly linked list O(n) and not O(1)?

Practical 5: Doubly Linked List – Insert & Delete

Unit: Unit 2 | Topic: Linked Lists

Aim	Implement a doubly linked list in C and perform insertion and deletion from both ends.

Objectives

- Understand the structure of a doubly linked list with prev and next pointers.
- Insert nodes at the beginning and end of the list.
- Delete nodes from the beginning and end.
- Traverse the list in both forward and backward directions.

Steps to Follow

20. Define a node with data, prev, and next pointers.
21. Implement insertAtBeginning() and insertAtEnd().
22. Implement deleteFromBeginning() and deleteFromEnd().
23. Implement forwardTraversal() and backwardTraversal().
24. Demonstrate with at least 4 nodes.

Reference Code (Study & Write in Notebook by Hand)

Note: Do NOT copy-paste. Understand the logic and write the complete program in your own handwriting.

```
#include<stdio.h>
#include<stdlib.h>
struct Node { int data; struct Node *prev, *next; };
struct Node* createNode(int val) {
    struct Node* n = malloc(sizeof(struct Node));
    n->data=val; n->prev=n->next=NULL;
    return n;
}
```

Expected Output

Forward and backward traversal of the list after each operation.

Viva Questions

Q1. How does a doubly linked list differ from a singly linked list in terms of memory?

Q2. What is the time complexity of inserting a node at the beginning of a doubly linked list?

Practical 6: Stack Implementation Using Array

Unit: Unit 2 | Topic: Stacks

Aim	Implement a stack using an array and perform PUSH, POP, and PEEK operations.

Objectives

- Understand the LIFO (Last In First Out) principle of a stack.
- Implement push() with overflow check.
- Implement pop() with underflow check.
- Implement peek() and display operations.

Steps to Follow

25. Define a stack with a fixed-size array and a top variable.
26. Implement push(), pop(), peek(), isEmpty(), isFull(), and display().
27. Use a menu-driven program to test all operations.
28. Handle overflow and underflow conditions with appropriate messages.

Reference Code (Study & Write in Notebook by Hand)

Note: Do NOT copy-paste. Understand the logic and write the complete program in your own handwriting.

```
#include<stdio.h>
#define MAX 10
int stack[MAX], top=-1;
void push(int val) {
    if(top==MAX-1) { printf("Stack Overflow!\n"); return; }
    stack[++top]=val;
}
int pop() {
    if(top==-1) { printf("Stack Underflow!\n"); return -1; }
    return stack[top--];
}
void display() {
    for(int i=top;i>=0;i--) printf("%d ",stack[i]);
    printf("\n");
}
```

Expected Output

Stack contents after each push and pop operation, with overflow/underflow messages.

Viva Questions

Q1. What real-world applications use a stack data structure?

Q2. What happens when you try to pop from an empty stack? How did you handle it?

Practical 7: Stack Application – Parenthesis Matching

Unit: Unit 2 | Topic: Stacks / Applications

Aim	Use a stack to check whether an expression has balanced parentheses.

Objectives

- Apply the stack concept to solve a real problem.
- Push opening brackets onto the stack and match closing brackets.
- Determine if an expression is balanced or unbalanced.

Steps to Follow

29. Read a string expression containing brackets: (), {}, [].
30. Scan the expression character by character.
31. Push every opening bracket onto the stack.
32. For every closing bracket, pop from the stack and check for a match.
33. At the end, if the stack is empty, the expression is balanced.

Reference Code (Study & Write in Notebook by Hand)

Note: Do NOT copy-paste. Understand the logic and write the complete program in your own handwriting.

```
#include<stdio.h>
#include<string.h>
char stack[100]; int top=-1;
int match(char open, char close) {
    return (open=='(' && close==')') ||
           (open=='{' && close=='}') ||
           (open=='[' && close==']');
}
int isBalanced(char expr[]) {
    for(int i=0; expr[i]; i++) {
        char c=expr[i];
        if(c=='('||c=='{'||c=='[') stack[++top]=c;
        else if(c==')'||c=='}'||c==']') {
            if(top==-1 || !match(stack[top],c)) return 0;
            top--;
        }
    }
    return top==-1;
}
```

Expected Output

"Balanced" or "Not Balanced" for at least 3 test expressions.

Viva Questions

Q1. Why is a stack the ideal data structure for this problem?

Q2. Test your program with: {[()]} and {[(])} — what is the output?

Practical 8: Queue Implementation Using Array

Unit: Unit 2 | Topic: Queues

Aim	Implement a queue using an array and perform ENQUEUE and DEQUEUE operations.

Objectives

- Understand the FIFO (First In First Out) principle of a queue.
- Implement enqueue() with overflow check.
- Implement dequeue() with underflow check.
- Implement display() to view the current queue contents.

Steps to Follow

34. Define a queue with a fixed-size array, front, and rear variables.
35. Implement enqueue(), dequeue(), isEmpty(), isFull(), and display().
36. Use a menu-driven program to test all operations.
37. Handle overflow and underflow conditions.

Reference Code (Study & Write in Notebook by Hand)

Note: Do NOT copy-paste. Understand the logic and write the complete program in your own handwriting.

```
#include<stdio.h>
#define MAX 10
int queue[MAX], front=-1, rear=-1;
void enqueue(int val) {
    if(rear==MAX-1) { printf("Queue Full!\n"); return; }
    if(front==-1) front=0;
    queue[++rear]=val;
}
int dequeue() {
    if(front==-1||front>rear) { printf("Queue Empty!\n"); return -1; }
    return queue[front++];
}
void display() {
    for(int i=front;i<=rear;i++) printf("%d ",queue[i]);
    printf("\n");
}
```

Expected Output

Queue contents after each enqueue and dequeue, with appropriate messages.

Viva Questions

Q1. What is the main limitation of a linear queue implemented using an array?

Q2. How does a circular queue solve the problem of wasted space?

Practical 9: Circular Queue Implementation

Unit: Unit 2 | Topic: Queues

Aim	Implement a circular queue using an array to overcome the limitation of a linear queue.

Objectives

- Understand the concept of circular arrangement in a queue.
- Implement enqueue and dequeue using modular arithmetic.
- Demonstrate that space freed by dequeue can be reused.

Steps to Follow

38. Implement a circular queue where (rear+1) % MAX wraps around.
39. Implement circularEnqueue() and circularDequeue().
40. Show that after dequeuing 3 elements, 3 new elements can be enqueued at those positions.
41. Display the queue after each operation.

Reference Code (Study & Write in Notebook by Hand)

Note: Do NOT copy-paste. Understand the logic and write the complete program in your own handwriting.

```
#include<stdio.h>
#define MAX 5
int cq[MAX], front=-1, rear=-1;
void enqueue(int val) {
    int next=(rear+1)%MAX;
    if(next==front) { printf("Circular Queue Full!\n"); return; }
    if(front==-1) front=0;
    rear=next; cq[rear]=val;
}
int dequeue() {
    if(front==-1) { printf("Queue Empty!\n"); return -1; }
    int val=cq[front];
    if(front==rear) { front=rear=-1; }
    else front=(front+1)%MAX;
    return val;
}
```

Expected Output

Demonstrate the circular reuse of space with at least 8 enqueue/dequeue operations.

Viva Questions

Q1. Why do we use (rear+1) % MAX instead of rear+1 in a circular queue?

Q2. What is the maximum number of elements a circular queue of size MAX can hold?

Practical 10: Stack Using Linked List

Unit: Unit 2 | Topic: Stacks / Linked Lists

Aim	Implement a dynamic stack using a singly linked list (no fixed size limit).

Objectives

- Understand how dynamic memory can be used to create a stack without size restriction.
- Implement push() by inserting at the head of the linked list.
- Implement pop() by deleting the head node.
- Compare this with array-based stack implementation.

Steps to Follow

42. Define a node structure with data and next pointer.
43. Implement push() to add a node at the top (head).
44. Implement pop() to remove the node at the top (head).
45. Implement display() to show the stack from top to bottom.
46. Demonstrate with at least 5 push and 2 pop operations.

Reference Code (Study & Write in Notebook by Hand)

Note: Do NOT copy-paste. Understand the logic and write the complete program in your own handwriting.

```
#include<stdio.h>
#include<stdlib.h>
struct Node { int data; struct Node* next; };
struct Node* top=NULL;
void push(int val) {
    struct Node* n=malloc(sizeof(struct Node));
    n->data=val; n->next=top; top=n;
}
int pop() {
    if(!top) { printf("Stack Empty!\n"); return -1; }
    int val=top->data;
    struct Node* tmp=top; top=top->next; free(tmp);
    return val;
}
```

Expected Output

Stack contents after each push and pop operation.

Viva Questions

Q1. What advantage does a linked list based stack have over an array based stack?

Q2. Is there a possibility of overflow in a linked list stack? Explain.

Practical 11: Queue Using Linked List

Unit: Unit 2 | Topic: Queues / Linked Lists

Aim	Implement a dynamic queue using a linked list where size is not fixed in advance.

Objectives

- Implement enqueue() by inserting at the rear of the linked list.
- Implement dequeue() by removing from the front of the linked list.
- Understand how front and rear pointers are maintained.
- Free memory after dequeue to prevent memory leaks.

Steps to Follow

47. Maintain two pointers: front and rear.
48. Implement enqueue() to add a node at rear.
49. Implement dequeue() to remove a node from front.
50. Display the queue after each operation.
51. Test with 5 enqueue and 3 dequeue operations.

Reference Code (Study & Write in Notebook by Hand)

Note: Do NOT copy-paste. Understand the logic and write the complete program in your own handwriting.

```
#include<stdio.h>
#include<stdlib.h>
struct Node { int data; struct Node* next; };
struct Node *front=NULL, *rear=NULL;
void enqueue(int val) {
    struct Node* n=malloc(sizeof(struct Node));
    n->data=val; n->next=NULL;
    if(!rear) { front=rear=n; return; }
    rear->next=n; rear=n;
}
int dequeue() {
    if(!front) { printf("Queue Empty!\n"); return -1; }
    int val=front->data;
    struct Node* tmp=front; front=front->next;
    if(!front) rear=NULL;
    free(tmp); return val;
}
```

Expected Output

Queue contents after each enqueue and dequeue.

Viva Questions

Q1. How is the rear pointer updated when the last element is dequeued?

Q2. What is the space complexity of a linked list queue compared to an array queue?

Practical 12: Mini Project – Student Record Management Using Linked List

Unit: Unit 1 & 2 | Topic: Arrays / Linked Lists (Integration)

Aim	Build a simple student record management system using a linked list to store and manage student data.

Objectives

- Apply linked list concepts to a real-world problem.
- Store student name, roll number, and marks in each node.
- Implement add, display, search by roll number, and delete by roll number.
- Display students in sorted order by marks.

Steps to Follow

52. Define a structure with: int roll; char name[30]; float marks; struct Node* next.
53. Implement addStudent() to insert at end.
54. Implement displayAll() to show all student records.
55. Implement searchByRoll() to find a student.
56. Implement deleteByRoll() to remove a student.
57. Implement sortByMarks() using Bubble Sort on the linked list.
58. Use a menu-driven program for all operations.

Reference Code (Study & Write in Notebook by Hand)

Note: Do NOT copy-paste. Understand the logic and write the complete program in your own handwriting.

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
struct Student {
    int roll;
    char name[30];
    float marks;
    struct Student* next;
};
// Implement: addStudent, displayAll,
// searchByRoll, deleteByRoll, sortByMarks
```

Expected Output

Complete menu-driven system with all operations working correctly.

Viva Questions

Q1. Why is a linked list a better choice than an array for this application?

Q2. How would you modify this program to also delete a student by name?

Submission Checklist

Before uploading on Google Classroom, make sure:

- ☐ Each practical is written neatly by hand in your notebook with: Practical No., Aim, Code, and Output.
- ☐ All 12 handwritten practicals are scanned or clearly photographed and saved as PDF/image files.
- ☐ Screenshot of the program output is taken for EACH of the 12 practicals.
- ☐ All Viva Question answers are written and compiled in a single Word/PDF document: VivaAnswers_RollNo.pdf

- ☐ Files are named correctly: Practical1_Scan_RollNo.pdf | Practical1_Output_RollNo.png | VivaAnswers_RollNo.pdf
- ☐ All files are uploaded to Google Classroom before the deadline.

All the Best! — HOD, Department of Computer Science, DHSK College